

QUEUES

BREADTH-FIRST TRAVERSAL

COMPLETE BINARY TREES

Problem Solving with Computers-II

C++

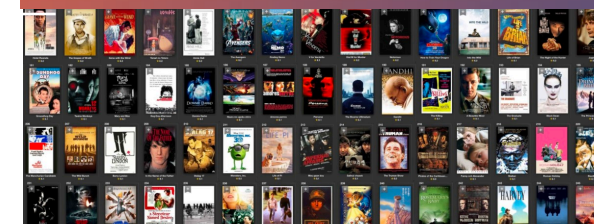
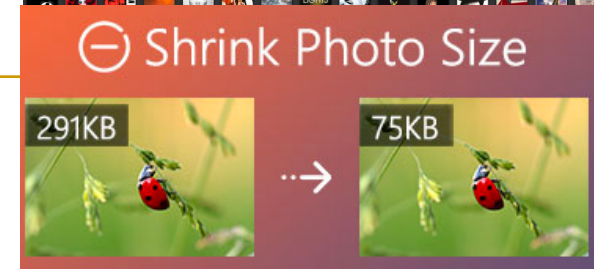
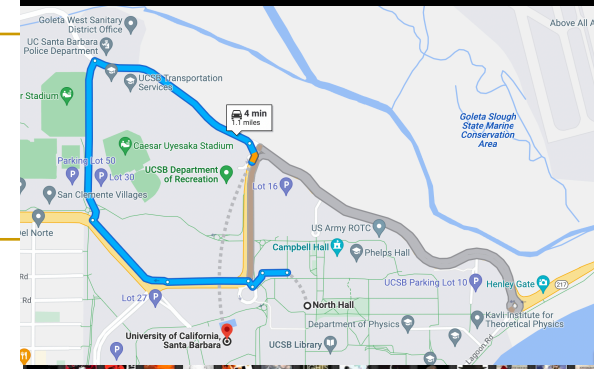
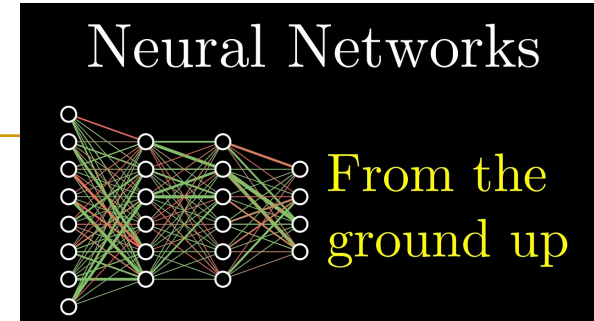
```
#include <iostream>
using namespace std;

int main(){
    cout<<"Hola Facebook\n";
    return 0;
}
```



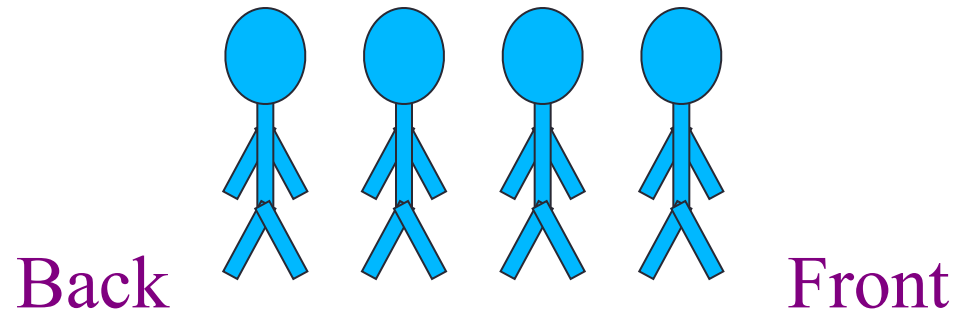
From Data Structures to Real-World Applications

Data Structure	Algorithm	Real-World Application
Queue	Breadth-First Search (BFS)	🤖 Machine Learning (PA03 : Prediction in NNs)
Queue	Round-Robin Scheduling	💻 Operating Systems (Task scheduling)
Priority Queue	Dijkstra's Algorithm	📍 GPS Navigation (Shortest path)
Priority Queue	Huffman Coding	💾 Data Compression (ZIP, JPEG, MP3)
Your choice!	You design!	Querying a movie data set (PA02)



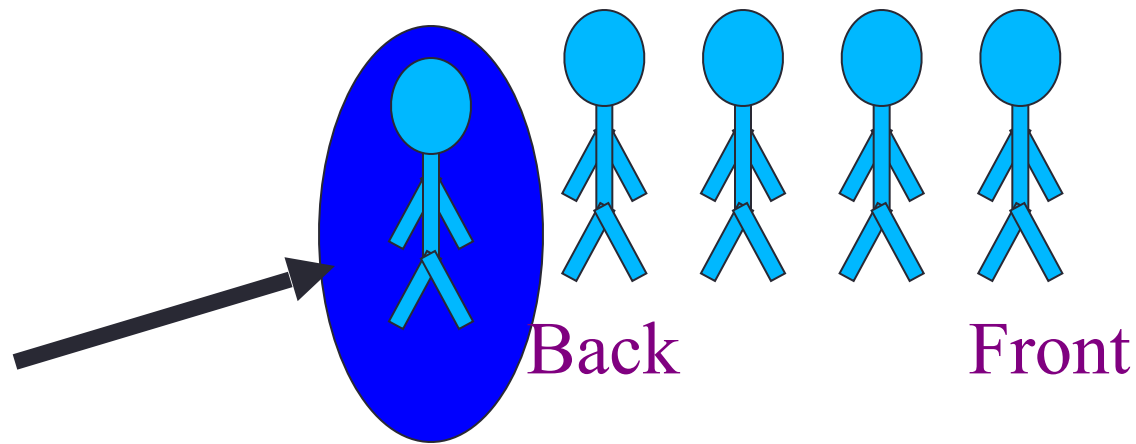
Queue

- A queue is like a queue of people waiting to be serviced
- The queue has a front and a back.



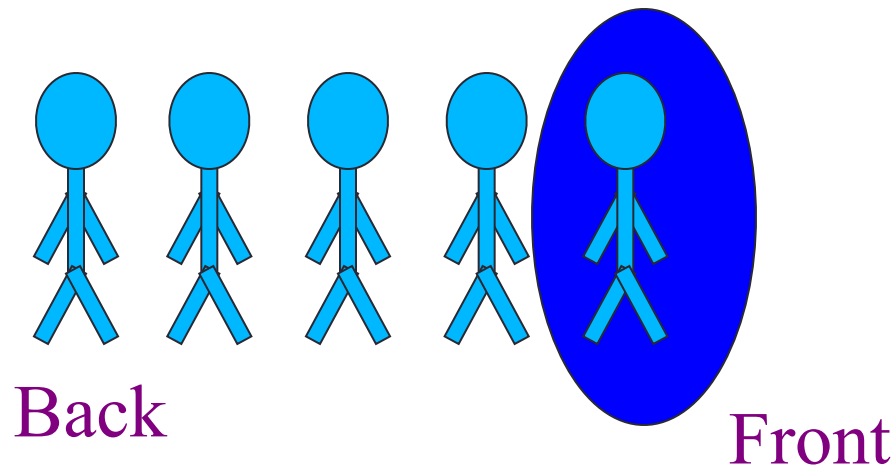
Queue Operations: push, pop, front, back

New people must enter the queue at the back. The C++ queue class calls this a push operation.



Queue Operations: push, pop, front, back

- To check the item in the front of the queue, use **front()**
- To check the item at the back of the queue, use **back()**
- When an item is taken from the queue, it always comes from the front. The C++ queue calls this a **pop**



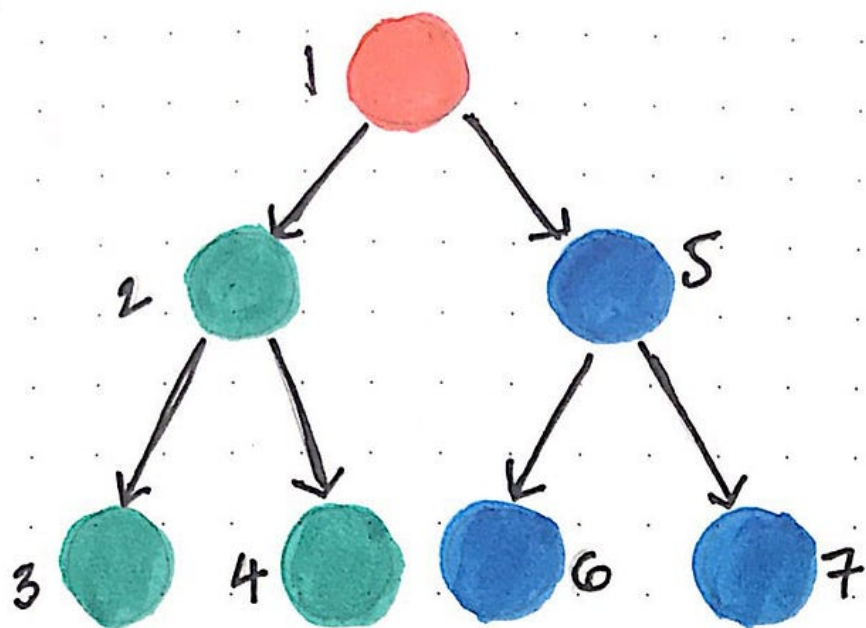
Queue and deque class

- The C++ standard template library has a queue template class: **std::queue**
 - **O(1) for all operations**
- C++ STL also offers a **Double-ended queue** called **std::deque**
 - insert and delete on both ends of the queue
 - **O(1) for all operations**

```
template <class Item>
class queue<Item>
{
public:
    queue( );
    void push(const Item& entry);
    void pop( );
    bool empty( ) const;
    Item front( ) const;
    Item back( ) const;

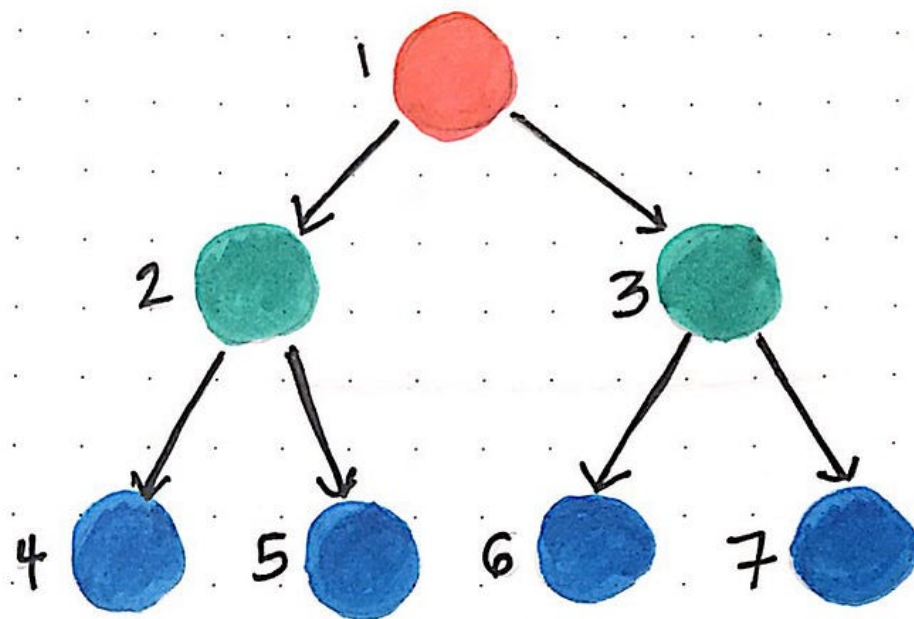
};
```

std::queue: <https://cplusplus.com/reference/queue/queue/?kw=queue>
std::deque: <https://cplusplus.com/reference/deque/deque/?kw=deque>



Depth-first search

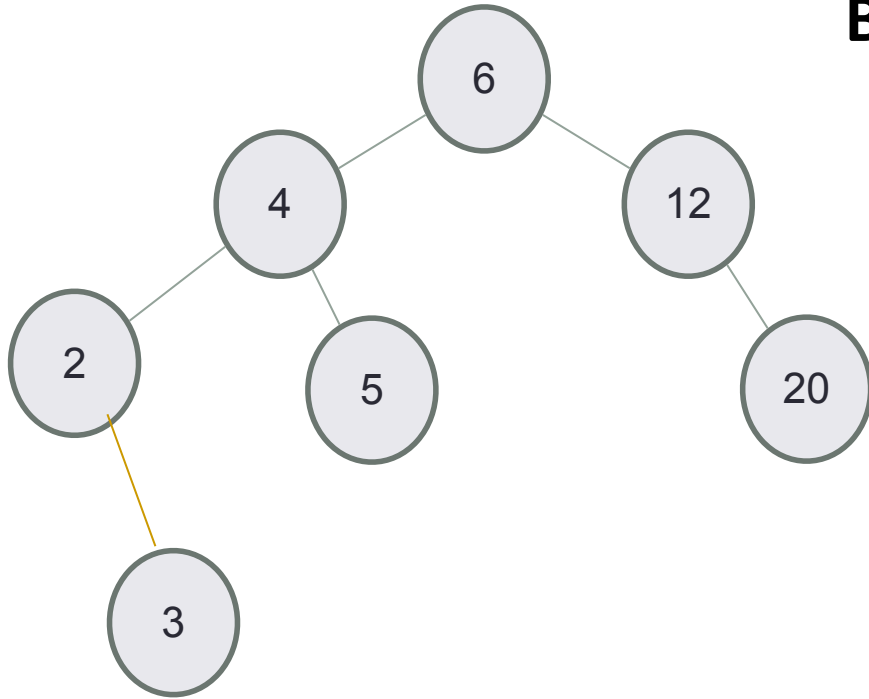
- Traverse through left subtree(s) first, then traverse through the right subtree(s).



Breadth-first search

- Traverse through one level of children nodes, then traverse through the level of grandchildren nodes (and so on...).

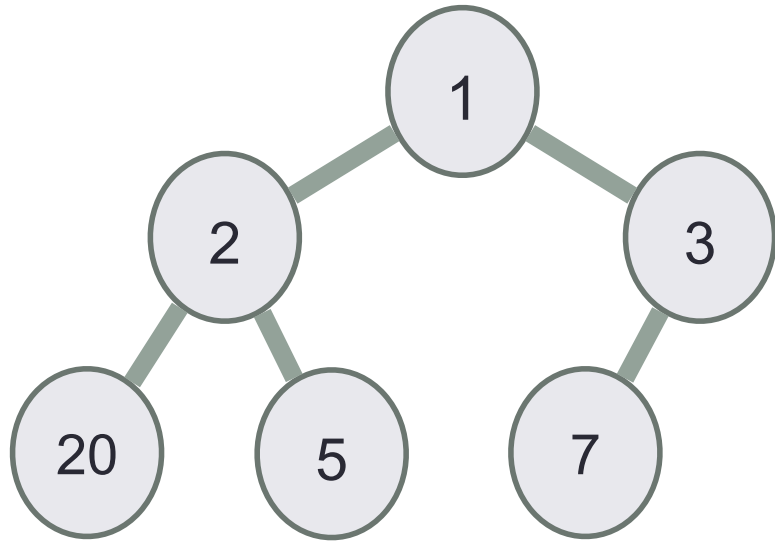
Breadth-first traversal/search



BFS Algo:

- Create an empty **queue**.
- Insert the **root** into the **queue**.
- While queue is not empty,
 - Print the key in the front of the queue
 - Insert all the children of the node into the queue.
 - Pop the front node from the queue

Breadth-first traversal

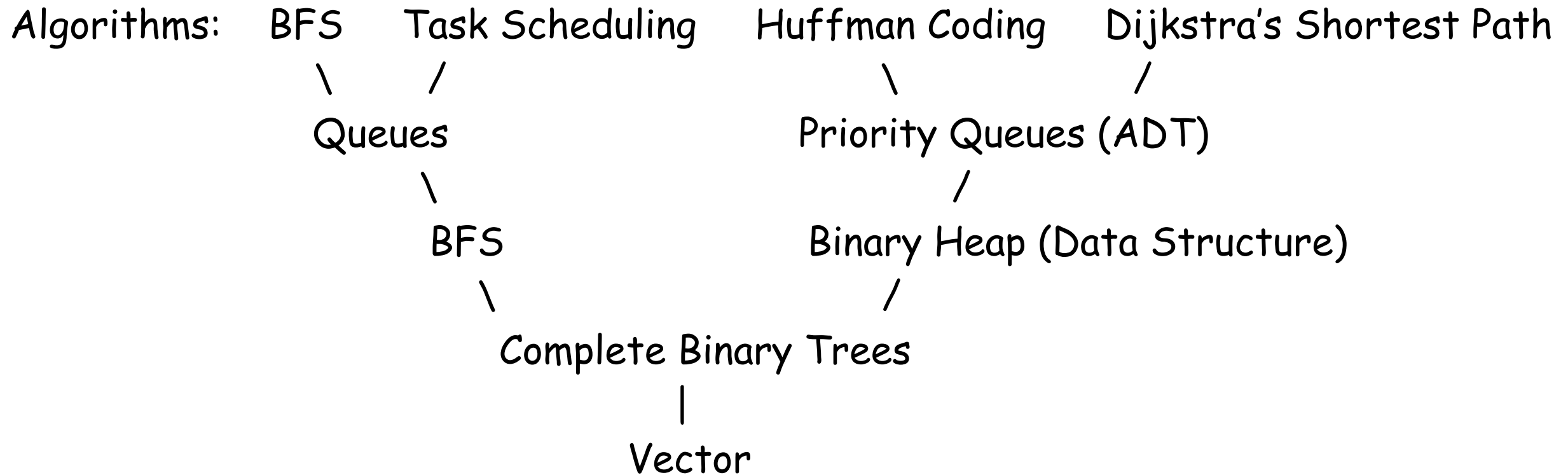


BFS Algo (store output in a vector: result):

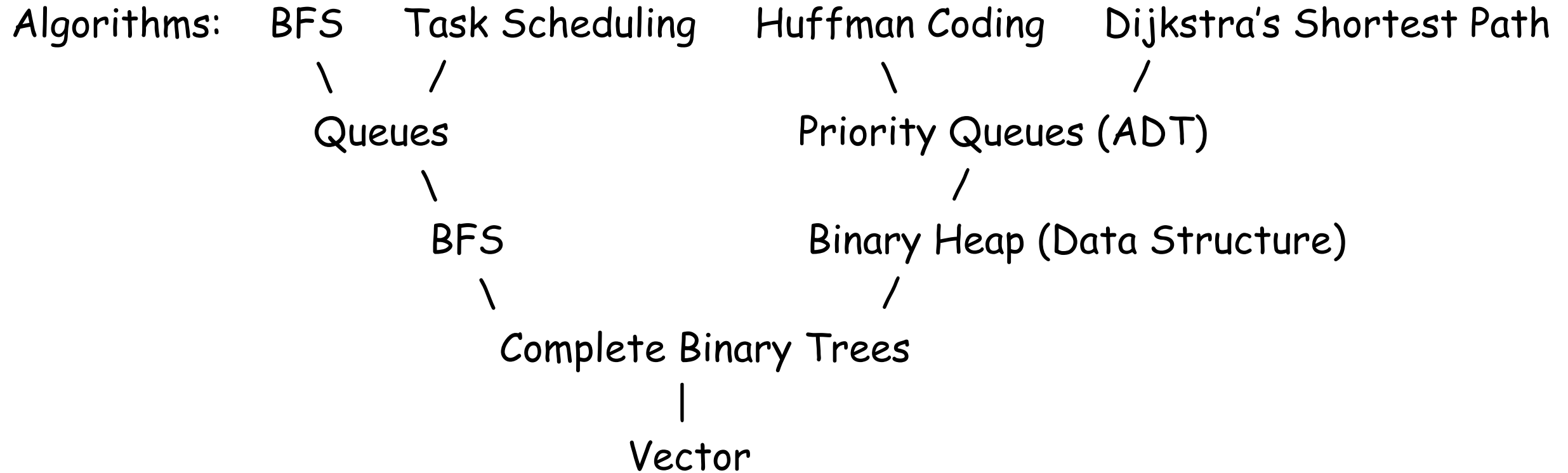
- Create an empty **queue**.
- Create an empty **vector** called **result**.
- Insert the **root** into the **queue**.
- While queue is not empty,
 - **Append the key in the front of the queue to result**
 - Insert all the children of the node into the queue.
 - Pop the front node from the queue

Trace BFS for the given tree. What is the resulting vector?

Interesting connections: from vectors to algorithms!



Interesting connections: from vectors to algorithms!

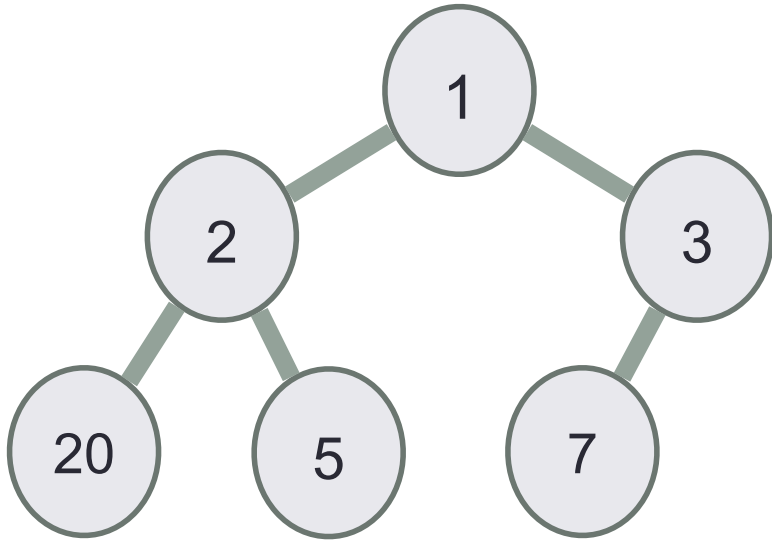


The `priority_queue` **abstract data type** is commonly implemented using a **binary heap**.

A **binary heap** is a specific kind of **complete binary tree**.

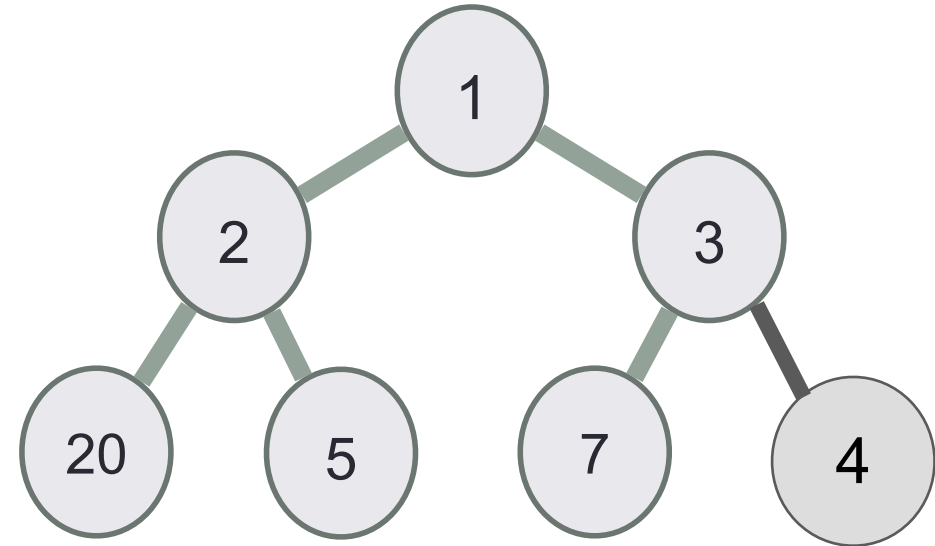
Complete binary trees are efficiently implemented using **vectors**, by indexing nodes in **BFS order**.

Types of Balanced Trees



Complete Binary Tree:

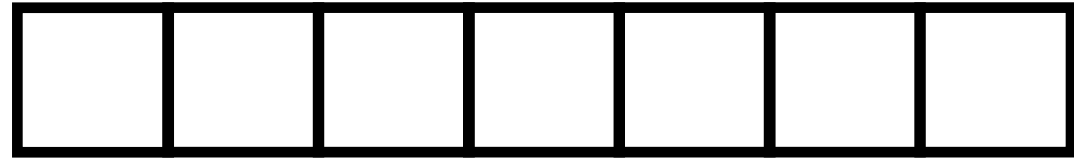
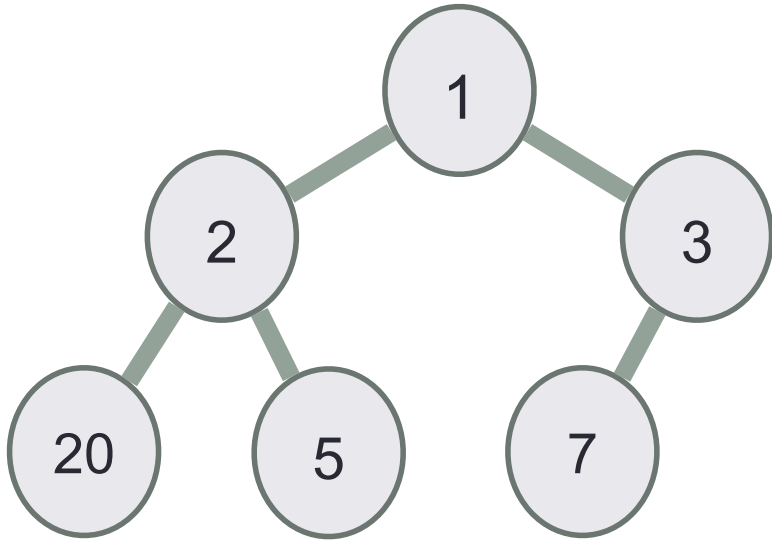
Every level is completely filled (except possibly the last level), and all nodes on the last level are as far left as possible



Full Binary Tree: A complete binary tree whose last level is completely filled

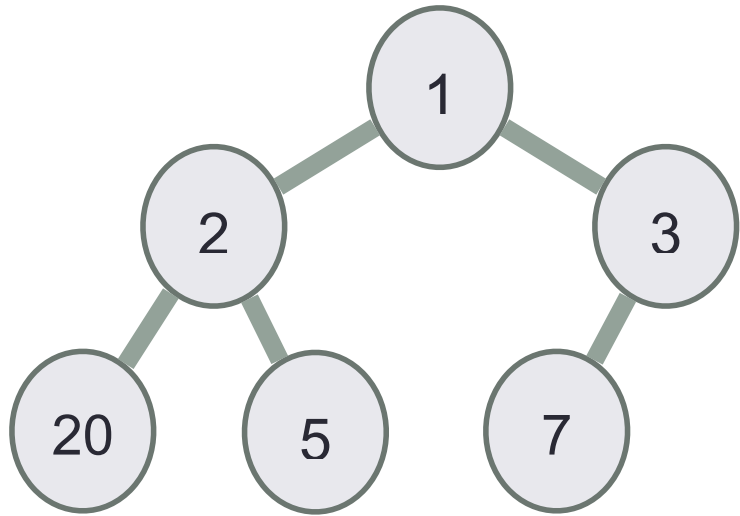
Complete/full binary trees are **balanced trees!**

Representing a complete binary tree as a vector!

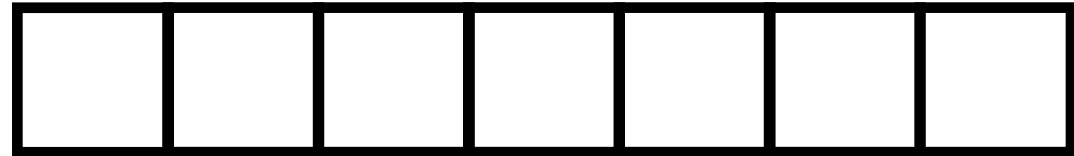


- How is the index of each key related to the index of its parent?
- How is the index of each key related to the indices of its left and right child?

Representing a complete binary tree as a vector!



index
key
parent
left child
right child



Root is at index 0

For a key at index i , index of its

- parent is $\lfloor (i - 1) / 2 \rfloor$
- left child is $2i + 1$
- right child is $2i + 2$

Traverse up the tree using the vector (only)!

Root is at index 0

For a key at index i , index of its

- parent is $\lfloor (i - 1)/2 \rfloor$
- left child is $2i + 1$
- right child is $2i + 2$

1	2	3	20	5	7	
---	---	---	----	---	---	--

Starting at the last node in the last level (7), write the indices of the keys visited on the path to the root node with key (1):

- A. 5, 4, 3, 2, 1, 0
- B. 5, 4, 2, 1, 0
- C. 5, 3, 1
- D. 5, 2, 0
- E. None of the above

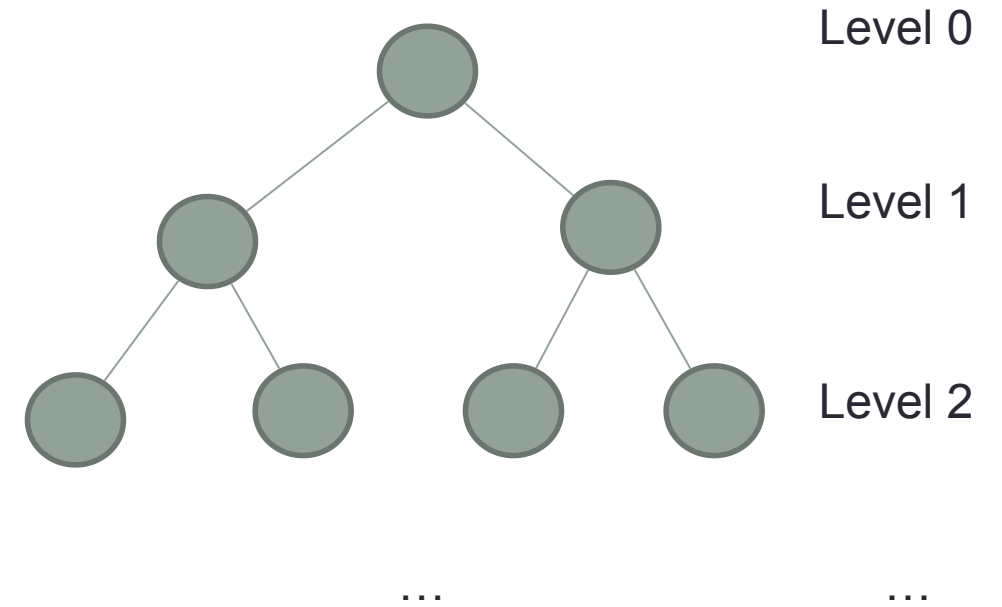
Add a new key with value 4 to the tree represented by this vector

1	2	3	20	5	7	
---	---	---	----	---	---	--

What is the complexity of adding new keys to a complete binary tree?

- A. $O(1)$**
- B. $O(\log n)$**
- C. $O(n)$**
- D. None of the above**

Show that a complete binary tree is balanced



Related Leetcode problems to attempt in problem set 3:

- Level Order Traversal of Binary Tree (medium): <<https://leetcode.com/problems/binary-tree-level-order-traversal/description/?envType=problem-list-v2&envId=binary-tree>>
- Binary Level Order Traversal II (medium): <<https://leetcode.com/problems/binary-tree-level-order-traversal-ii/description/?envType=problem-list-v2&envId=binary-tree>>